

An Energy, Delay and Priority-Aware Task Offloading Algorithm for Fog Computing

Thanmayee Pounjula, Bhargavi Ravirala and Sanjaya Kumar Panda, *Senior Member, IEEE*

Department of Computer Science and Engineering

National Institute of Technology Warangal, Warangal - 506004, Telangana, India

{pounju_911936, ravira_911941}@student.nitw.ac.in, sanjaya@nitw.ac.in

Abstract—Internet of Things (IoT), fog and cloud computing are the most widespread technologies that have become a part of our day-to-day activities. The integration of these technologies will become crucial components for the future Internet as their usage and acceptance rapidly increase. IoT devices (or terminal nodes (TNs)) have limited resource capability to carry out all of their generated tasks. Therefore, they depend on the cloud to assist them in completing all their tasks. However, the distance between TNs to the cloud may lead to network congestion and uneven delay. As a result, fog nodes (FNs) work as an intermediary between TNs and the cloud to minimize delay in completing the TNs' tasks. In this context, previous studies assign the TNs' tasks to FNs based on various criteria, namely energy, delay and priority among the tasks, without combining them. Fair task offloading (FTO) recently combines these criteria to assign the TNs tasks to FNs without significantly considering load balancing among FNs. This paper introduces a multi-objective task offloading algorithm, called EDP-TO, by considering all the criteria and load balancing. The proposed algorithm uses the multi-objective function to select the FNs for offloading. It divides the tasks into multiple sub-tasks and assigns them to the selected FNs, such that the overall delay is minimized. The performance of the proposed algorithm is shown with and without load balancing, called EDP-TO-LB and EDP-TO-WLB, and compared with FTO by considering three scenarios. The comparison results show the superiority of the EDP-TO over the FTO in terms of energy, delay and fairness.

Index Terms—Fog Computing, Task Offloading, Energy, Delay, Priority, Terminal Node, Fog Node.

I. INTRODUCTION

The fifth-generation (5G) technology is expected to connect numerous intelligent devices, such as autonomous vehicles, e-healthcare equipment, smart home devices, and wearable devices, in crowded areas and provides better speed than the existing technologies [1], [2]. Moreover, smartphone growth increases exponentially and is expected to reach billion in numbers [3]. IoT provides a platform to connect these intelligent devices, owned by the end users, over the Internet. However, these devices have limited computational and storage capabilities and battery power. Therefore, these devices cannot process all the generated tasks on their own and rely on the cloud. Cloud provides a platform to execute these tasks by fulfilling the requirements in terms of various services offered under the infrastructure as a service (IaaS), platform as a service (PaaS) and software as a service (SaaS). It also allows the end users to concentrate solely on their tasks/applications

rather than managing the resources [4]. Therefore, it provides the end users flexibility, scalability and cost-effectiveness.

The massive growth of IoT devices/TNs generates more tasks and needs to be processed timely. However, due to TNs' resource limitations, some tasks/sub-tasks cannot be processed in the TNs' timely. Therefore, they are pushed to the cloud to fulfil their requirements [5]. Unfortunately, the overall delay is increased as the cloud is very far away from the TNs [6]. It is noteworthy to mention that the delay comprises computation and communication processes between the TNs and the cloud. Although the cloud provides massive computation power, the communication delay associated with the cloud (i.e., network congestion, bandwidth and others) is relatively more. Therefore, fog is introduced as an intermediary between the TNs and the cloud to minimize the overall delay. It provides services from the network edge and solves the problem of timely processing of TNs tasks. The TNs can get services from the FNs instead of directly obtaining them from the cloud. However, in case of unavailability of services, the TNs can get the services from the cloud through the FNs. Alternatively, TNs rely on cloud and fog computing to fulfill the real-world demands. Therefore, integrating IoT, fog, and cloud is essential for solving day-to-day problems, namely fire detection and fighting, patient monitoring, emergency services and many more.

The process of transferring tasks/sub-tasks from the TNs to the FN(s) in a fog computing environment is called as task offloading. It is noteworthy to mention that dividing tasks into multiple sub-tasks and offloading them to different FNs can improve the quality of service [6]–[8]. There are various criteria to consider while offloading these tasks: energy consumption, task delay, the priority of FNs and load balancing among all the FNs in the network. On the other hand, an appropriate decision should be taken to optimize the above-mentioned criteria. One such decision is offloading the TNs task to a single FN or a set of FNs, whereas another decision is to transfer the entire task or sub-tasks. Previous studies have considered one or more criteria and taken the decision to offload TNs tasks. To our knowledge, most of the studies have not considered all these criteria. Recently, FTO combines energy, delay and priority to assign the TNs tasks to FNs. However, load balancing is not considered while assigning the TNs tasks to FNs. This phenomenon inspires us to develop a new offloading algorithm that makes a balance between

energy, delay and priority and ensures load balancing among the FNs.

This paper presents the problem of offloading the M TNs' tasks to the N FNs where $M \gg N$ and offers a solution towards the same. We propose a multi-objective algorithm, called an energy, delay and priority-aware task offloading (EDP-TO) algorithm for a fog computing environment. This proposed algorithm considers the energy, task delay and priority of the FNs to prevent overburdening of FNs. It consists of three steps. (1) A list of FNs is selected based on the multi-objective function. (2) The selected FNs are examined for load balancing. In case of overburdening of FNs, the list of selected FNs is revised. (3) The tasks are divided into sub-task and offloaded to the selected nodes to minimize the overall task delay. The proposed algorithm is examined with or without load balancing, called EDP-TO-LB and EDP-TO-WLB. The performance of the proposed algorithm is compared with FTO using three scenarios. In each scenario, we varied the number of FNs and TNs in the range of 5 to 30 and 50 to 3000, respectively. The comparison results demonstrate that the proposed algorithm performs well in terms of energy, delay and fairness.

The paper has the following contribution to the literature.

- 1) We develop a task offloading algorithm, which uses a multi-objective function to select the FNs. The function takes the linear combination of energy, delay and priority and their corresponding weight.
- 2) We consider the load balancing among the FNs while selecting them to offload the TNs' tasks.
- 3) We consider heterogeneous TNs and FNs to make the proposed algorithm realistic.

The highlight of the upcoming sections is as follows. Section II presents the related work on task offloading. Section III offers the task offloading model for a fog computing environment and problem statement. Section IV elaborates on the proposed algorithm with its step-by-step explanation. Section V presents the simulation results of the existing and proposed algorithm and their comparison. Finally, Section VI summarises the paper with future works.

II. RELATED WORK

Many task offloading algorithms have been proposed by individually aiming at energy [9], delay [9] and priority. Such algorithms may create a significant burden on certain FNs if load balancing is not taken into consideration. Moreover, the potential FNs can be failed and become unavailable for the TNs. Therefore, the load balancing of these FNs is challenging and needs to be studied.

Meng, Wang and Zhang [9] have discussed a hybrid offloading problem by incorporating computation and communication powers. They consider two destinations, namely cloud servers and fog servers, to offload the tasks. The objective is to minimize the energy consumption. However, they have not considered minimizing task delay, prioritizing FNs, and balancing the load. Deng et al. [10] have introduced a dynamic scheduling algorithm based on the workload. They aim to

maximize the average throughput and ensure task delay in the worst case. The algorithm is studied from the optimality perspective. However, their algorithm is limited to a two-layer architecture without considering the cloud layer.

Baccarelli et al. [11] have integrated the Internet of everything with fog computing and introduced the fog of everything architecture. They have shown the performance of energy and delay through the simulation process. However, they consider the fog of everything migration theoretically without showing it practically. Zhang et al. [7] have presented a task offloading algorithm by considering three factors, namely energy consumption, historical average energy and priority. They consider a fairness metric based on these factors before offloading the TNs tasks to a single FN. The algorithm is a two-step process. In the first step, they check the feasibility of task offloading to each FN in the fog network. The second step offloads the task to a single FN based on the fairness metric. Their algorithm does not offload to multiple FNs.

Recently, Zhang et al. [12] have considered the fair offloading framework to offload the tasks to multiple FNs. The framework models the energy consumption, task delay and priority of FNs as a fairness metric. In this context, they have proposed a FTO algorithm comprising two steps. In the first step, it selects the FNs using a fairness metric. In the next step, the tasks are offloaded to selected FNs, which results in minimum task delay. They have not explicitly considered load balancing among FNs.

The proposed algorithm has the following differences from the above literature.

- 1) Unlike [7], [12], the proposed algorithm, EDP-TO, balance the load among the FNs after the selection of appropriate FNs.
- 2) Unlike [7], [12], The proposed algorithm considers a multi-objective function to balance the trade-off among energy, delay and priority.
- 3) Like FTO [12], the EDP-TO deals with multiple FNs. But, it considers heterogeneous TNs and FNs compared to FTO.

III. SYSTEM MODEL AND PROBLEM FORMULATION

This section introduces the mathematical representation of the system model in a heterogeneous fog computing environment followed by the problem formulation.

A. System Model

The system model is designed for task offloading services. We use a set of notations to explain the model, which is listed in Table I. The major components of this model are TNs and FNs, which are distributed randomly, as shown in Fig. 1. Note that these nodes are heterogeneous by nature. TN generates a computational task with size b and selects n FNs (i.e., $n \ll N$) based on various criteria, such as historical mean energy consumption, delay, priority and many more. Then the task b is divided into multiple sub-task of sizes, i.e., $\{b_j^{TN}, b_1, b_2, \dots, b_n\}$, in such a way that the overall task delay is minimized. Note that b_j^{TN} is the size of sub-task computed by the TN

itself and $\{b_1, b_2, \dots, b_n\}$ are the sub-task sizes computed by n selected FNs. The TN delay is only due to the computation of the sub-task with size b_j^{TN} (i.e., d_j^{TN}), and the FN_i delay is due to the computation and communication of the sub-task with size b_i (i.e., d_i). Mathematically [12],

$$b = b_j^{TN} + \sum_{i=1}^n b_i \quad (1)$$

The task delay is the maximum time required to complete all the $n + 1$ sub-task. Mathematically [12],

$$\text{delay} = \max(d_j^{TN}, d_1, d_2, \dots, d_n) \quad (2)$$

where

$$d_j^{TN} = \frac{b_j^{TN}}{c_j^{TN}} \quad (3)$$

and

$$d_i = \frac{b_i}{t_{ji}} + \frac{b_i}{c_i} = \frac{b_i}{r_i}, 1 \leq i \leq n \quad (4)$$

where

$$r_i = \left(\frac{1}{t_{ji}} + \frac{1}{c_i} \right)^{-1} \quad (5)$$

Here, $\max$ is a pre-defined function to find the maximum, $\{d_j^{TN}, d_1, d_2, \dots, d_n\}$ are the delays in TN and selected FNs, respectively, c_j^{TN} is the computation rate of TN_j , b_i is the size of sub-task offloaded to FN_i , t_{ji} is the rate of data transmission between the TN_j and FN_i , c_i is the computation rate of FN_i and r_i is the processing rate of FN_i .

Once these tasks are successfully completed in the FNs, the results are returned to the TNs. As the result of the sub-task is a small packet, the transmission time of the result is negligible. Further, TNs consolidate the results based on the end-user requirements.

The energy consumed by the TN_j (i.e., e_j^{TN}) is calculated by adding the energy consumed for local computation and data transmission to the n FNs. It is mathematically expressed as follows.

$$e_j^{TN} = \frac{b_j^{TN}}{c_j^{TN}} \times cp_j^{TN} + \sum_{i=1}^n \frac{b_i}{t_{ji}} \times tp_{ji} \quad (6)$$

where cp_j^{TN} is the computation power of TN_j and tp_{ji} is the transmission power between the TN_j and FN_i .

The energy consumed by FN_i (i.e., e_i) is stated as follows.

$$e_i = \frac{b_i}{c_i} \times cp_i \quad (7)$$

where cp_i is the computation power of FN_i .

The overall energy consumed to process a computation task with size b (i.e., e_{total}) is defined as follows.

$$e_{total} = e_j^{TN} + \sum_{i=1}^n e_i \quad (8)$$

As FNs have different computational capabilities, TNs tasks should not always be assigned to those with high computational power. If so, the energy consumption of those FNs is

TABLE I: Key notations and their description

Notation	Unit	Description
b	bit	TN's task size
n	-	Number of FNs selected for task offloading
b_i	bit	Size of sub-task offloaded to FN_i , $1 \leq i \leq n$
b_j^{TN}	bit	Size of sub-task computed by TN_j , $1 \leq j \leq m$
N	-	Number of FNs present in the network
t_{ji}	bits/s	Rate of data transmission between the TN_j and FN_i
p_i	-	Priority of the FN_i , $1 \leq i \leq n$
c_i	bits/s	Computation rate of FN_i , $1 \leq i \leq n$
c_j^{TN}	bits/s	Computation rate of TN_j , $1 \leq j \leq m$
r_i	bits/s	Processing rate of FN_i , $1 \leq i \leq n$
cp_i	Watts	Computation power of FN_i , $1 \leq i \leq n$
cp_j^{TN}	Watts	Computation power of TN_j , $1 \leq j \leq m$
tp_{ji}	Watts	Transmission power between the TN_j and FN_i
e_i	mJ	Energy consumed by FN_i , $1 \leq i \leq n$
$\bar{e}_i$	mJ	Historic mean energy of FN_i , $1 \leq i \leq n$

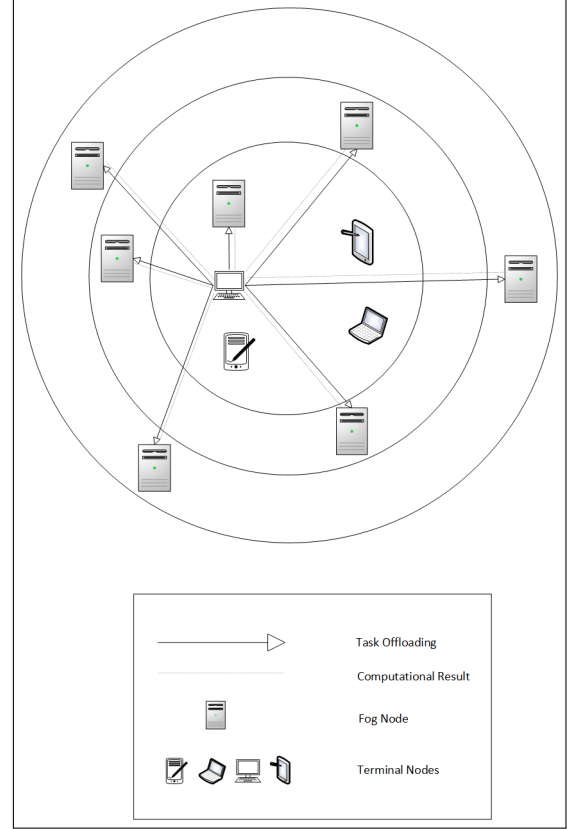


Fig. 1: A system architecture.

increased to a greater extent. On the other hand, FNs with low computational power may increase the task delay. In this paper, we need to balance energy and task delay by developing a suitable task offloading strategy. The above-discussed system model is similar to [12].

B. Problem Formulation

Consider a set of M TNs, $TN = \{TN_1, TN_2, TN_3, \dots, TN_M\}$ and a set of N FNs, $FN = \{FN_1, FN_2, FN_3, \dots, FN_N\}$. Each TN has a computing rate in Mbps and computing power in Watts. Each FN also has a computing rate in Mbps, computing power in Watt and priority. Note that the priority is of two types, namely active and passive. The communication

between the TNs and FNs is represented by transmission rate in Mbps and transmission power in Watts. The task offloading problem is to select the suitable FNs for each TN task, such that the following objectives are satisfied.

- 1) The energy consumption of FNs is minimized.
- 2) The task delay is minimized.
- 3) The load balancing among the FNs is achieved.

The task offloading problem is restricted to the following constraints.

- 1) A task can be assigned to one or more FNs.
- 2) A FN cannot migrate the load to another FN.
- 3) The task offloading is limited to single-hop communication.

IV. PROPOSED ALGORITHM

This section presents the proposed algorithm, EDP-TO, for a heterogeneous fog computing environment. The objectives of the algorithm are to minimize energy consumption and task delay. It also targets to balance the load among the FNs. For this, the algorithm follows a three-step process. The first step selects n FNs among the N FNs of the heterogeneous network under consideration using a multi-objective function. Note that $n \ll N$ and n FNs are selected for a single computational task. The multi-objective function f for each FN depends on energy, delay and priority and is simply defined as follows.

$$f = \lambda_1 \times \left(\frac{1}{\text{energy}}\right) + \lambda_2 \times \left(\frac{1}{\text{delay}}\right) + (1 - \lambda_1 - \lambda_2) \times (\text{priority}) \quad (9)$$

However, it can be expressed mathematically as follows.

$$f = \lambda_1 \times \left(\frac{1}{\bar{e}_i}\right) + \lambda_2 \times r_i + (1 - \lambda_1 - \lambda_2) \times p_i \quad (10)$$

where $0 \leq \lambda_1, \lambda_2 \leq 1$, $\lambda_1 + \lambda_2 \leq 1$, $\bar{e}_i$ is the historic mean energy of FN_i and p_i is the priority of FN_i . Note that $p_i = 1$ if the FN is passive and $p_i = 2$ if it is an active FN. The value of $\frac{1}{\bar{e}_i}$, r_i and p_i is normalized between 0 and 1 to avoid dominance of one against others.

Next, the proposed algorithm chooses the top n FNs with high function values. The function value shows the trade-off between energy, delay and priority. In the next step, the selected FNs are examined for load balancing. For this, the waiting time of each FN is determined. If any FN is idle, then that FN is chosen for offloading by replacing one of the selected FNs. Otherwise, the selected FN of the first step is chosen for offloading. Note that the initial waiting time of FNs is zero. It is noteworthy to mention that the proposed algorithm is examined with and without involving this step to show the need for load balancing and call them EDP-TO-LB and EDP-TO-WLB, respectively.

The last step divides the task into $n + 1$ sub-task and offloads to the selected FNs in such a way that the overall task delay is minimized. The overall task delay is the maximum delay of all the $n + 1$ sub-task. Mathematically,

$$\min_{b_j^{TN}, b_1, b_2, \dots, b_n} [\max(d_j^{TN}, d_1, d_2, \dots, d_n)] \quad (11)$$

where $\min$ is a pre-defined function to find the minimum.

As the task delay of FN depends on the ratio of sub-task size offloaded to that FN and the processing rate of that FN, and the processing rate of that FN is constant, the task delay is directly related to sub-task size. The overall delay can be minimized if all sub-task delays are the same [12]. Mathematically,

$$d^{optimal} = d_j^{TN} = d_1 = d_2 = \dots = d_n \quad (12)$$

where $d^{optimal}$ is the optimal sub-task delay. From Eq. 3, and Eq. 4, we can rewrite Eq. 12 as follows.

$$d^{optimal} = \frac{b_j^{TNoptimal}}{c_j^{TN}} = \frac{b_1^{optimal}}{r_1} = \frac{b_2^{optimal}}{r_2} = \dots = \frac{b_n^{optimal}}{r_n} \quad (13)$$

Eq. 1 can be re-written as follows.

$$b = b_j^{TNoptimal} + \sum_{i=1}^n b_i^{optimal} \quad (14)$$

where

$$b_i^{optimal} = r_i \frac{b_j^{TNoptimal}}{c_j^{TN}} \quad (15)$$

From Eq. 13, we can rewrite Eq. 14 as follows.

$$b = b_j^{TNoptimal} + r_1 \frac{b_j^{TNoptimal}}{c_j^{TN}} + r_2 \frac{b_j^{TNoptimal}}{c_j^{TN}} + \dots + r_n \frac{b_j^{TNoptimal}}{c_j^{TN}} \quad (16)$$

Eq. 16 can be simplified as follows.

$$b = b_j^{TNoptimal} \left(1 + \frac{r_1}{c_j^{TN}} + \frac{r_2}{c_j^{TN}} + \dots + \frac{r_n}{c_j^{TN}}\right) \quad (17)$$

Eq. 17 can be further simplified as follows.

$$b_j^{TNoptimal} = b \left(1 + \sum_{i=1}^n \frac{r_i}{c_j^{TN}}\right)^{-1} \quad (18)$$

If we substitute Eq. 18 in Eq. 15, then it can be represented as follows.

$$b_i^{optimal} = r_i \frac{b \left(1 + \sum_{i=1}^n \frac{r_i}{c_j^{TN}}\right)^{-1}}{c_j^{TN}} \quad (19)$$

The optimal delay is obtained from Eq. 13 and Eq. 18 as follows.

$$d^{optimal} = \frac{b_j^{TNoptimal}}{c_j^{TN}} = \frac{b \left(1 + \sum_{i=1}^n \frac{r_i}{c_j^{TN}}\right)^{-1}}{c_j^{TN}} \quad (20)$$

The optimal energy consumption of TN_j can be calculated using Eq. 6, Eq. 18 and Eq. 19, which is shown as follows.

$$\begin{aligned} e_j^{TNoptimal} &= \frac{b_j^{TNoptimal}}{c_j^{TN}} \times c p_j^{TN} + \sum_{i=1}^n \frac{b_i^{optimal}}{t_{ji}} \times t p_{ji} \\ &= \frac{b \left(1 + \sum_{i=1}^n \frac{r_i}{c_j^{TN}}\right)^{-1}}{c_j^{TN}} \times c p_j^{TN} + \sum_{i=1}^n \frac{r_i \frac{b \left(1 + \sum_{i=1}^n \frac{r_i}{c_j^{TN}}\right)^{-1}}{c_j^{TN}}}{t_{ji}} \times t p_{ji} \end{aligned} \quad (21)$$

The optimal energy consumption of FN_i can be calculated using Eq. 7 and Eq. 19, which is shown as follows.

$$e_i^{optimal} = \frac{b_i^{optimal}}{c_i} cp_i = \frac{r_i \frac{b(1+\sum_{i=1}^n \frac{r_i}{c_i^{TN}})^{-1}}{c_j^{TN}}}{c_i} cp_i \quad (22)$$

The overall optimal energy consumed to process a computation task with size b (i.e., $e_{total}^{optimal}$) is defined as follows.

$$e_{total}^{optimal} = e_j^{TNoptimal} + \sum_{i=1}^n e_i^{optimal} \quad (23)$$

Once a task is offloaded, the algorithm recalculates the historic mean energy of selected FNs (i.e., $\bar{e}'_i$). It can be represented as follows [13], [14].

$$\bar{e}'_i = (1 - \alpha) \times \bar{e}_i + (\alpha) \times e_i \quad (24)$$

where α is a value between 0 and 1. In a similar way, $\bar{e}_j^{TN}$ and $\bar{e}$ can be calculated.

Jain's fairness index (F) is used to assess the fairness of the task offloading algorithm, which is defined as follows [15].

$$F = \frac{1}{N} \times \frac{\left(\sum_{i=1}^N \bar{e}_i\right)^2}{\sum_{i=1}^N \bar{e}_i^2} \quad (25)$$

If $F = 1$, the historic mean energy consumed by all the FNs is the same. The pseudo-code of the proposed algorithm is shown in Algorithm 1 in which Procedure 1 (i.e., EDP-TO-LB) is called. However, this procedure is not called in EDP-TO-WLB.

The time complexity of the proposed algorithm is $O(MN \log(N))$, where M and N are the number of TNs and FNs, respectively. However, the space complexity of the proposed algorithm is $O(MN)$.

V. SIMULATION RESULTS AND COMPARISON

This section presents the simulation results and compares the evaluation of the proposed algorithm EDP-TO (i.e., EDP-TO-LB and EDP-TO-WLB) and FTO algorithm [12] in terms of energy consumption, task delay, and fairness. The simulation parameters are shown in Table II. We assume that the correlation coefficient is one between the following pairs. (1) c_i and cp_i (2) c_j^{TN} and cp_j^{TN} (3) t_{ji} and tp_{ji} . It is noted that the above values follow Gaussian distributions and their mean (μ) and standard deviation (σ) are shown in Table II. Each FN's initial historic mean energy consumption is calculated as the average of the transmission rate from all TNs tasks to that FN.

We consider three scenarios to examine the performance of EDP-TO-LB, EDP-TO-WLB and FTO algorithms. In each scenario, we took different FNs and TNs. The number of FNs is five, and the number of TNs is 10 to 3000 in scenario 1, whereas the number of FNs is 15 and TNs is 30 to 3000 in scenario 2. On the contrary, the number of FNs is 30, and TNs is 50 to 3000 in scenario 3. However, the number of selected FNs is 2, 6 and 12, respectively, in three scenarios. Moreover,

Algorithm 1 EDP-TO

Inputs: FN_i , TN_j , t_{ji} , tp_{ji} , c_i , cp_i , c_j^{TN} , cp_j^{TN} , total historic mean energy ($\bar{e}$), TN historic mean energy ($\bar{e}_j^{TN}$), $\bar{e}_i$, p_i , α , N , n and M

Outputs: Mean task delay ($\bar{d}$), $\bar{e}$, F , $\bar{e}_i$ and $\bar{e}_j^{TN}$

- 1: Initialize the waiting time of each FN to zero
 - 2: **while** there is a task generated by any TN **do**
 - 3: Calculate r_i for each FN as shown in Eq. 5
 - 4: Normalize the value of $\frac{1}{\bar{e}_i}$, r_i and p_i
 - 5: Apply multi-objective function f for all the FNs as shown in Eq. 10
 - 6: Sort all the FNs based on the value of f in non-increasing order
 - 7: Choose top n FNs from N FNs
 - 8: Call Procedure *EDP-TO-LB* with waiting time of FNs
 - 9: Divide the task size into $n + 1$ sub-task sizes in an optimal way as shown in Eq. 18 and Eq. 19
 - 10: Determine the task delay $d_j^{optimal}$ and the energy consumption, $e_j^{TNoptimal}$, $e_i^{optimal}$ and $e^{optimal}$, as shown in Eq. 20, Eq. 21, Eq. 22 and Eq. 23, respectively
 - 11: Update the waiting time of selected FNs by adding the task delay
 - 12: Update $\bar{e}_j^{TN}$, $\bar{e}_i$ and $\bar{e}$ using Eq. 24
 - 13: **end while**
 - 14: Calculate $\bar{d} = \frac{1}{M} \sum_{j=1}^M d_j$, where d_j is the overall delay of task generated by TN_j
 - 15: Calculate F using Eq. 25
-

Procedure 1 EDP-TO-LB

- 1: Find the FNs with zero waiting time and include them to a new list of FNs
 - 2: **if** the number of FNs in new list $< n$ **then**
 - 3: Choose remaining FNs from the selected FNs in the non-increasing order of their f value and append to the new list
 - 4: **end if**
 - 5: Return the new list as selected FNs
-

60% of the FNs are active nodes in all three scenarios, i.e., 3, 9 and 18.

We compare the performance of EDP-TO-LB, EDP-TO-WLB and FTO algorithms in terms of total historic mean energy consumption, TN historic mean energy consumption and the average of FNs historic mean energy consumption of three scenarios, as shown in Fig. 2 to Fig. 10, respectively. It is evident from the figures that EDP-TO-LB and EDP-TO-WLB algorithms outperform the FTO algorithm. In EDP-TO-LB and EDP-TO-WLB algorithms, when the number of TNs increases, the historic mean energy consumption is less compared to FTO. The improvement is because the proposed algorithm considers multi-objective function and load balancing among the FNs.

TABLE II: Simulation Parameters

Parameter	Value
α	0.002
N	[5 ~ 30]
b	0.01 Mbits
M	[10 ~ 3000]
c_i	$\mu = 5, \sigma = 2$
cp_i	$\mu = 1, \sigma = 0.3$
c_j^{TN}	$\mu = 2, \sigma = 3$
cp_j^{TN}	$\mu = 0.5, \sigma = 2$
t_{ji}	$\mu = 4, \sigma = 3$
tp_{ji}	$\mu = 0.5, \sigma = 3$

The proposed and existing algorithms are also compared regarding mean task delay and fairness in three scenarios, as shown in Fig. 11 and Fig. 16, respectively. We can observe the supremacy of the EDP-TO-LB and EDP-TO-WLB algorithms over the FTO algorithm. Especially, when the number of TNs is more, the mean task delay is decreased, and the fairness of the task offloading is increased. This is due to the fair distribution of load among the FNs.

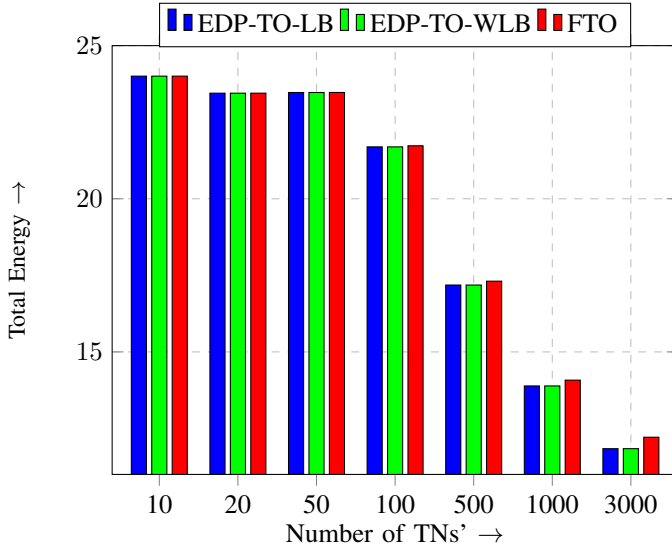


Fig. 2: Comparison of the total historic mean energy consumption in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 5.

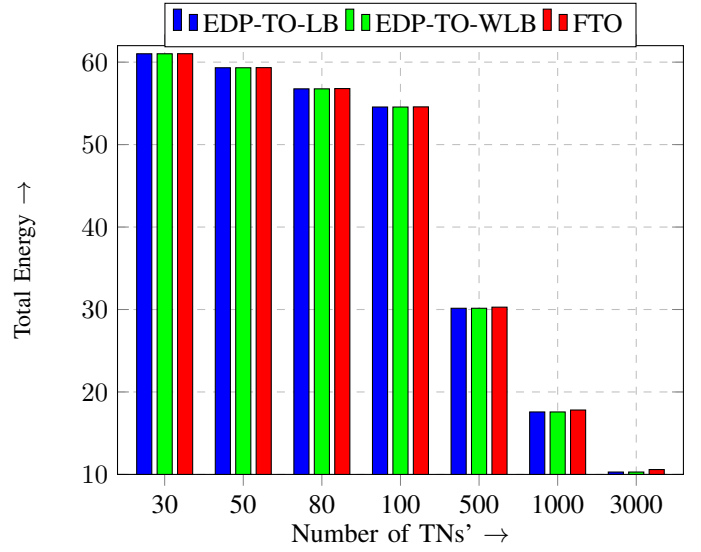


Fig. 3: Comparison of the total historic mean energy consumption in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 15.

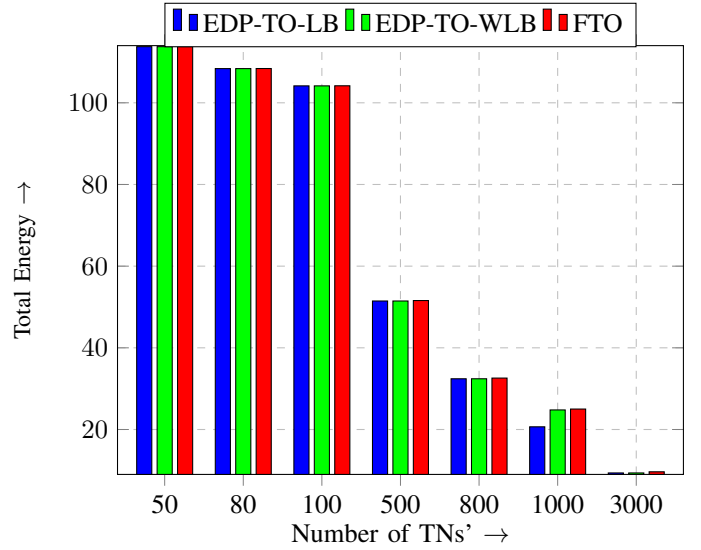


Fig. 4: Comparison of the total historic mean energy consumption in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 30.

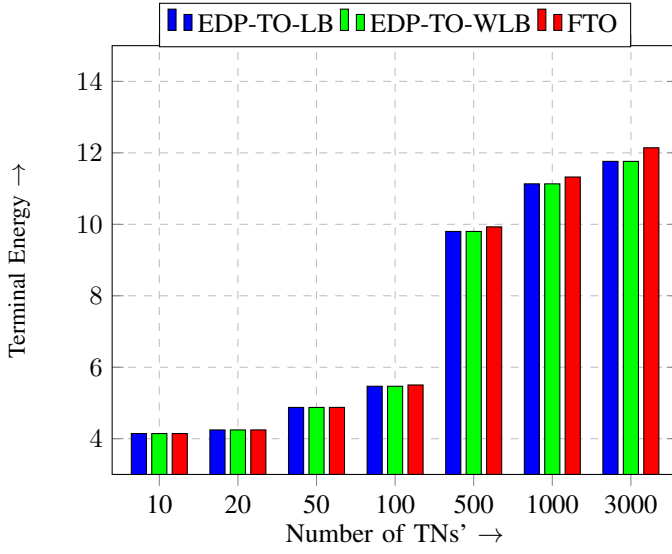


Fig. 5: Comparison of the TN historic mean energy in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 5.

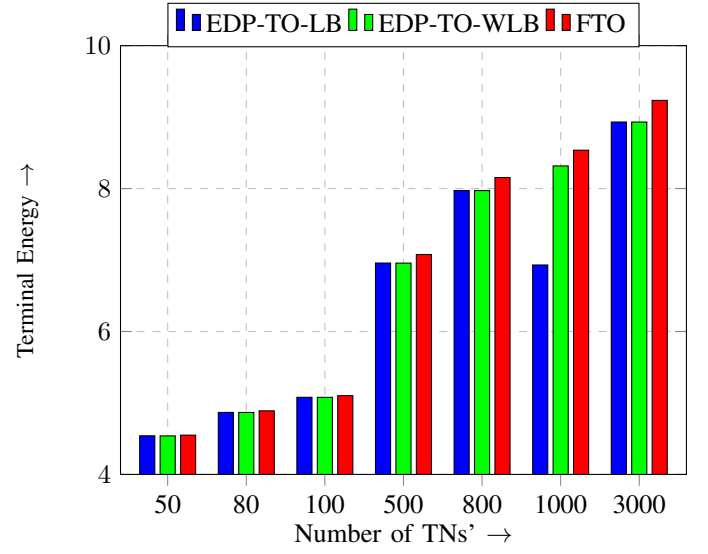


Fig. 7: Comparison of the TN historic mean energy in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 30.

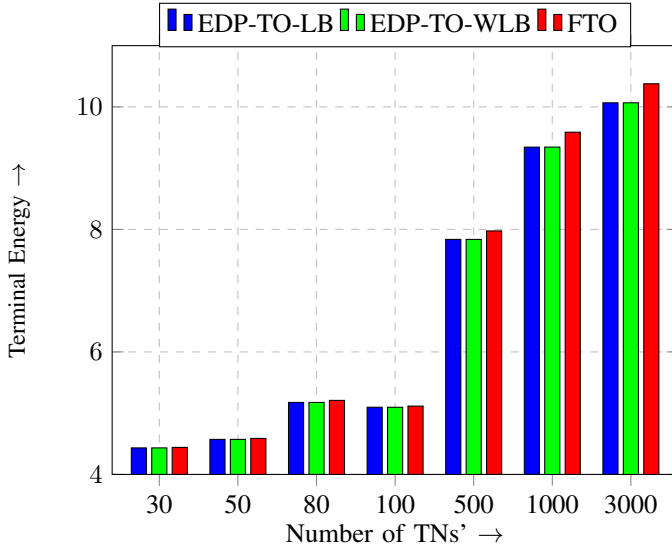


Fig. 6: Comparison of the TN historic mean energy in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 15.

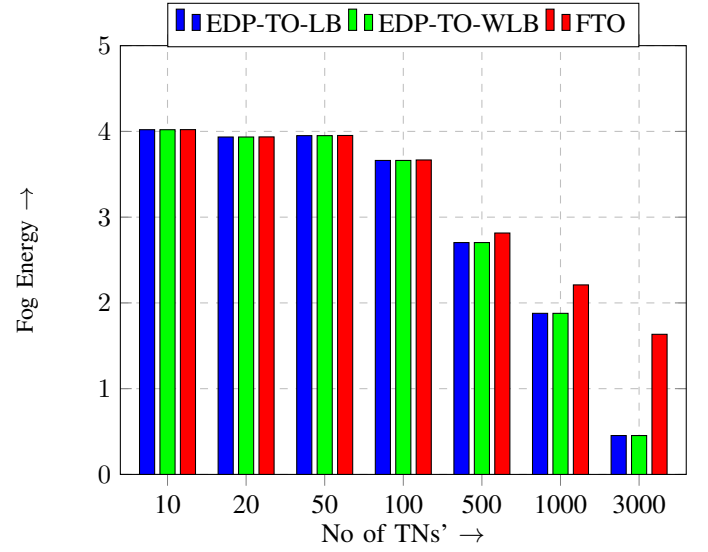


Fig. 8: Comparison of the the average of FNs historic mean energy consumption in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 5.

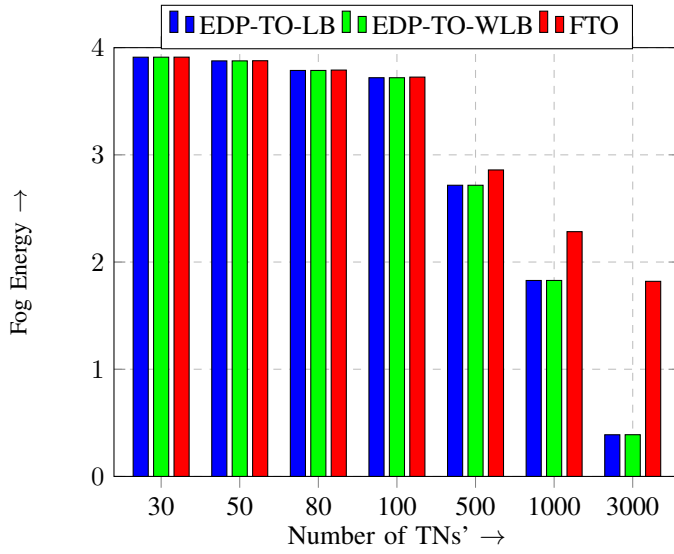


Fig. 9: Comparison of the the average of FNs historic mean energy consumption in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 15.

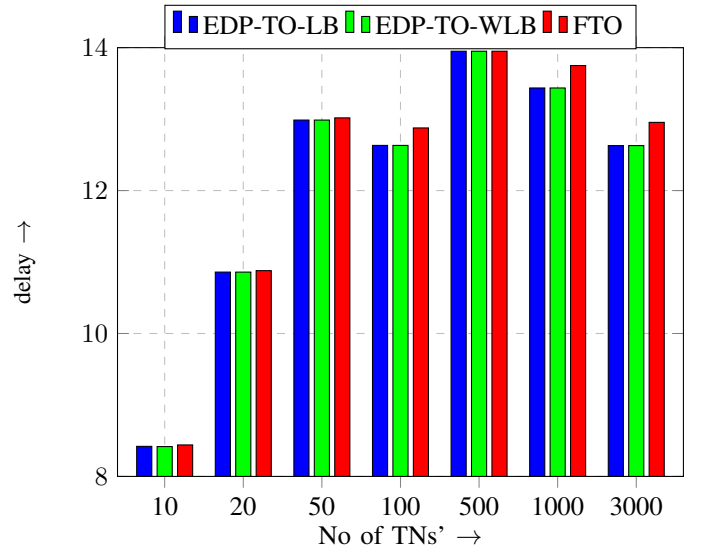


Fig. 11: Comparison of the mean task delay in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 5.

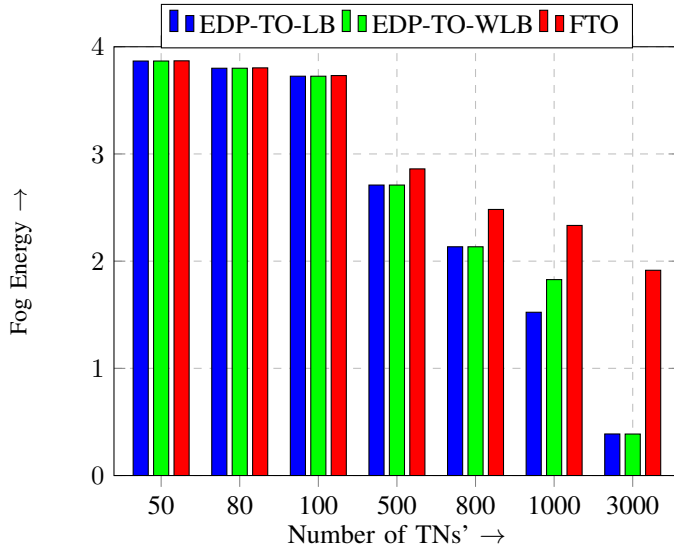


Fig. 10: Comparison of the the average of FNs historic mean energy consumption in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 30.

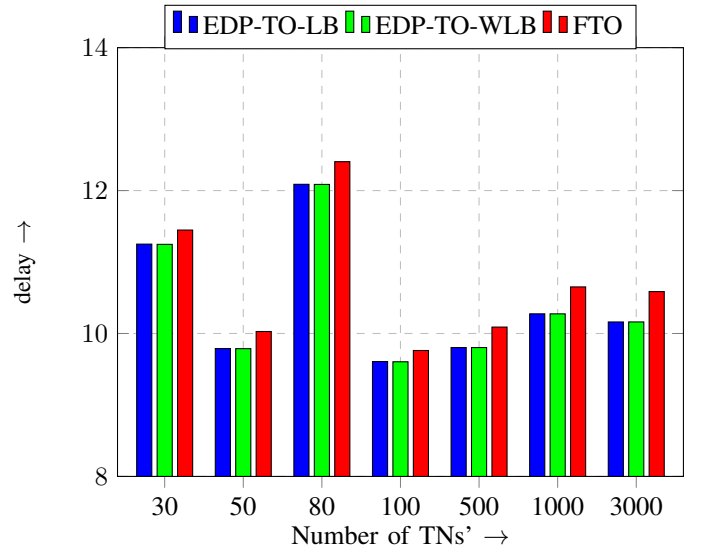


Fig. 12: Comparison of the mean task delay in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 15.

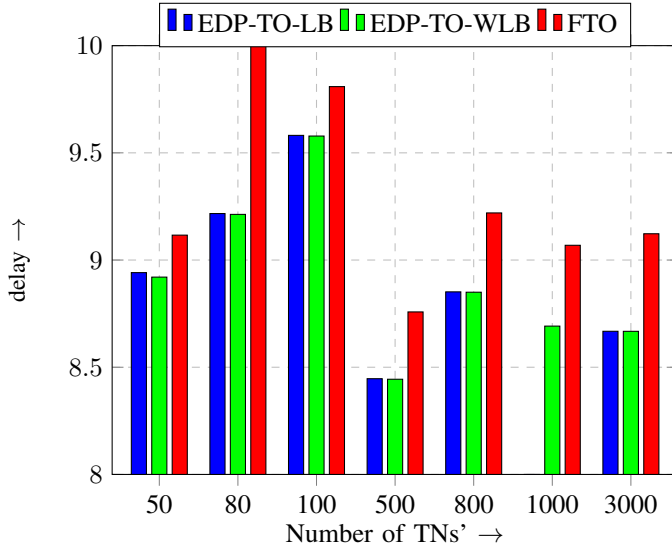


Fig. 13: Comparison of the mean task delay in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 30.

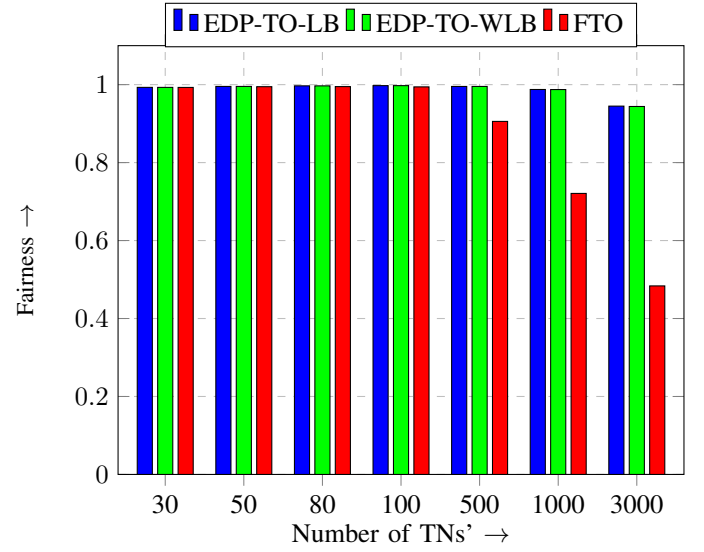


Fig. 15: Comparison of the fairness in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 15.

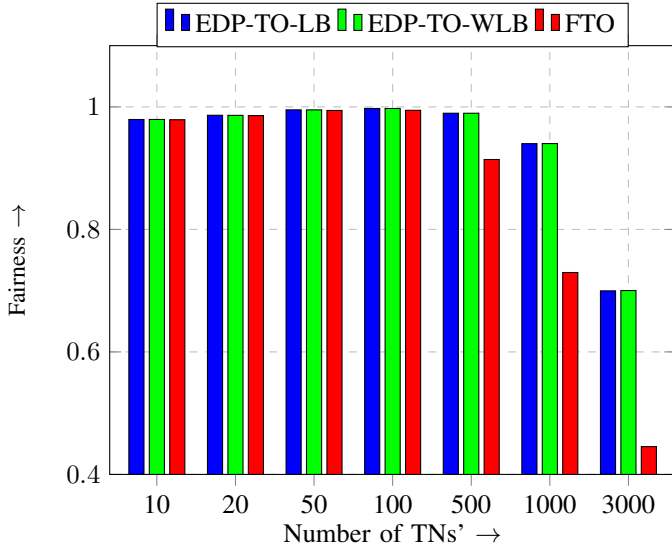


Fig. 14: Comparison of the fairness in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 5.

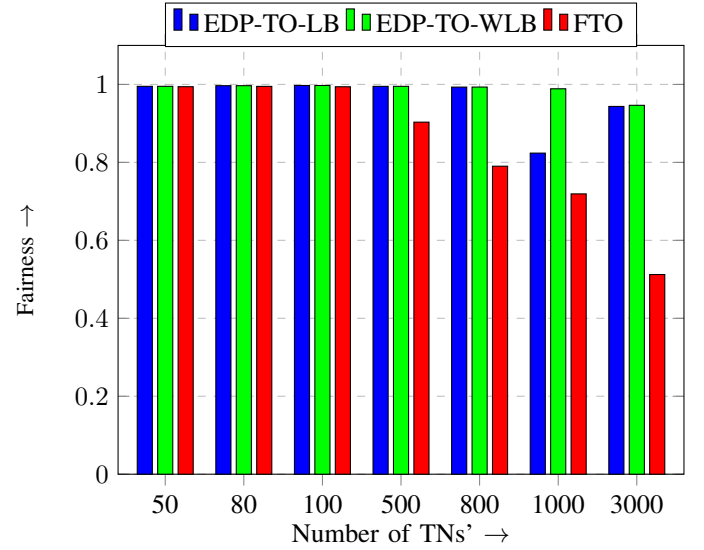


Fig. 16: Comparison of the fairness in EDP-TO-LB, EDP-TO-WLB and FTO algorithms for different number of TNs for FN = 30.

VI. CONCLUSION AND FUTURE WORK

This paper presents a task offloading algorithm, EDP-TO, for a heterogenous fog computing environment. It aims to minimize the energy and delay, and tries to balance the load among the FNs. The proposed algorithm consists of three steps. The first step selects the FNs based on a multi-objective function, the second step performs the load balancing, and the third step divides the task and offloads to the selected FNs. The offloading algorithm has time and space complexities of $O(MN \log(n))$ and $O(MN)$, respectively. The two versions of the proposed algorithm, namely EDP-TO-LB and EDP-TO-

WLB, are compared with the existing FTO algorithm in three scenarios and three performance metrics. Compared to the FTO algorithm, the proposed algorithm results in less historic mean energy consumption, less mean task delay and a higher fairness score. However, the proposed algorithm considers stationary TNs and FNs. It can be further extended by considering mobile TNs and FNs. Moreover, communication between FNs can be included in further research for better load balancing.

REFERENCES

- [1] Jayavardhana Gubbi, Rajkumar Buyya, Slaven Marusic, and Marimuthu Palaniswami. Internet of things (iot): A vision, architectural elements, and future directions. *Future generation computer systems*, 29(7):1645–1660, 2013.
- [2] Xiaohu Ge, Jing Yang, Hamid Gharavi, and Yang Sun. Energy efficiency challenges of 5g small cell networks. *IEEE communications Magazine*, 55(5):184–191, 2017.
- [3] Krupa B Doshi, Seong Hyun Moon, Michael D Whitaker, and Thurmon E Lockhart. Assessment of gait and posture characteristics using a smartphone wearable system for persons with osteoporosis with and without falls. *Scientific Reports*, 13(1):538, 2023.
- [4] Jeongho Kwak, Yeongjin Kim, Joohyun Lee, and Song Chong. Dream: Dynamic resource and task allocation for energy minimization in mobile cloud systems. *IEEE Journal on Selected Areas in Communications*, 33(12):2510–2523, 2015.
- [5] Alessio Botta, Walter De Donato, Valerio Persico, and Antonio Pescape. Integration of cloud computing and internet of things: a survey. *Future generation computer systems*, 56:684–700, 2016.
- [6] Mung Chiang, Sangtae Ha, Fulvio Rizzo, Tao Zhang, and I Chih-Lin. Clarifying fog computing and networking: 10 questions and answers. *IEEE Communications Magazine*, 55(4):18–20, 2017.
- [7] Guowei Zhang, Fei Shen, Zening Liu, Yang Yang, Kunlun Wang, and Ming-Tuo Zhou. Femto: Fair and energy-minimized task offloading for fog-enabled iot networks. *IEEE Internet of Things Journal*, 6(3):4388–4400, 2018.
- [8] Xu Chen, Lei Jiao, Wenzhong Li, and Xiaoming Fu. Efficient multi-user computation offloading for mobile-edge cloud computing. *IEEE/ACM transactions on networking*, 24(5):2795–2808, 2015.
- [9] Xianling Meng, Wei Wang, and Zhaoyang Zhang. Delay-constrained hybrid computation offloading with cloud and fog computing. *IEEE Access*, 5:21355–21367, 2017.
- [10] Yiqin Deng, Zhigang Chen, Deyu Zhang, and Ming Zhao. Workload scheduling toward worst-case delay and optimal utility for single-hop fog-iot architecture. *IET Communications*, 12(17):2164–2173, 2018.
- [11] Enzo Baccarelli, Paola G Vinueza Naranjo, Michele Scarpiniti, Mohammad Shojafar, and Jemal H Abawajy. Fog of everything: Energy-efficient networked computing architectures, research challenges, and a case study. *IEEE access*, 5:9882–9910, 2017.
- [12] Guowei Zhang, Fei Shen, Yang Yang, Hua Qian, and Wei Yao. Fair task offloading among fog nodes in fog computing networks. In *2018 IEEE international conference on communications (ICC)*, pages 1–6. IEEE, 2018.
- [13] Frank Kelly. Charging and rate control for elastic traffic. *European transactions on Telecommunications*, 8(1):33–37, 1997.
- [14] Francesco Capozzi, Giuseppe Piro, Luigi Alfredo Grieco, Gennaro Boggia, and Pietro Camarda. Downlink packet scheduling in lte cellular networks: Key design issues and a survey. *IEEE communications surveys & tutorials*, 15(2):678–700, 2012.
- [15] Dah Ming Chiu. A quantitative measure of fairness and discrimination for resource allocation in shared computer systems. Technical report, Digital Equipment Corporation, 1984.